Introduction to Specialization

1. Introduction to Data Science with Python Specialization

This course is the first of five in an intermediate-level Data Science specialization using Python. It focuses on applied data science skills and assumes basic knowledge of programming and statistics.

Python is chosen because it is easy to learn, widely taught, full-featured, and has strong data science libraries within the SciPy ecosystem. Tools like Jupyter Notebooks, Pandas, and Matplotlib form the foundation for later topics such as machine learning, text mining, and network analysis.

The course has four modules:

- Python fundamentals and prerequisites, including some optional advanced Python concepts.
- The pandas toolkit for working with tabular data, inspired by ideas from R and relational databases.
- Advanced pandas techniques such as boolean masking and hierarchical indexing.
- A course project focused on cleaning, merging, and analyzing real datasets, along with basic statistical methods.

The goal is to build practical skills in turning messy data into meaningful insights and to prepare learners for advanced

data science topics.

2. Introduction to the Course

Nothing to Note!!!!

3. The Coursera Jupyter Notebook System

Introduction to the coursera jupyter system.

4. Python Functions(ipynb)

Python Basics and Jupyter Notebooks Overview

This module gives a high-level introduction to Python, enough for learners with prior programming experience to succeed in the specialization. Beginners are encouraged to take an introductory course like *Python for Everybody* by Dr. Chuck Severance.

Python code in the course is run through Coursera using in-video prompts and Jupyter Notebooks. Jupyter allows code

to be written and executed in cells, stored persistently, and run without local installation. Notebooks are stateful, meaning variables persist across cells, but changes require re-execution. The restart and run all option resets and reruns everything.

Python is a high-level, interpreted, and interactive language, well suited for exploration, data cleaning, and investigation. It is dynamically typed, so variables can change types and type errors are handled at runtime.

Python syntax uses minimal boilerplate, no semicolons, and no variable declarations. Whitespace and indentation define scope. Functions are defined with def, use indentation, and can return values or implicitly return None. Functions support default and labeled parameters, enabling flexible calls without overloading.

Functions can also be assigned to variables and passed around, enabling basic functional programming concepts that will be explored later.

5 Python Types and Sequences(ipynb)

Python Built-in Types and Collections

Python is dynamically typed, but every object has a type that

can be inspected using the type() function. Common types include strings, None, integers, floats, and functions. Objects have associated data and methods.

Python centers heavily around collection types. The three main native collections are tuples, lists, and dictionaries.

Tuples are ordered and immutable collections, defined with parentheses, and can contain mixed data types. Lists are similar but mutable, defined with square brackets, and support operations like appending, indexing, looping, and length checks. Both lists and tuples are iterable and support common operations like len, min, max, concatenation with +, repetition with *, and membership testing with in.

Indexing in Python starts at zero and supports slicing. Slicing allows extraction of sub-sequences using start and end positions, supports negative indexing from the end, and allows omitted bounds. Strings are sequences of characters, so all slicing operations apply directly to them. Slicing is a core Python feature and is especially important in scientific computing.

Strings support concatenation, repetition, membership testing, and basic text processing. The split() method breaks strings into lists based on simple patterns, which is useful for basic text analysis. More advanced pattern matching is handled later with regular expressions.

Dictionaries are unordered, labeled collections that store key-

value pairs, defined with curly braces. Keys and values can be of any type. Dictionary elements are accessed and added using keys, and dictionaries can be iterated over by keys, values, or both using the items() method.

Python supports unpacking, where values from a tuple or list can be assigned to multiple variables in a single statement, as long as the number of variables matches the number of items.